DBMS PROJECT GROUP – 27

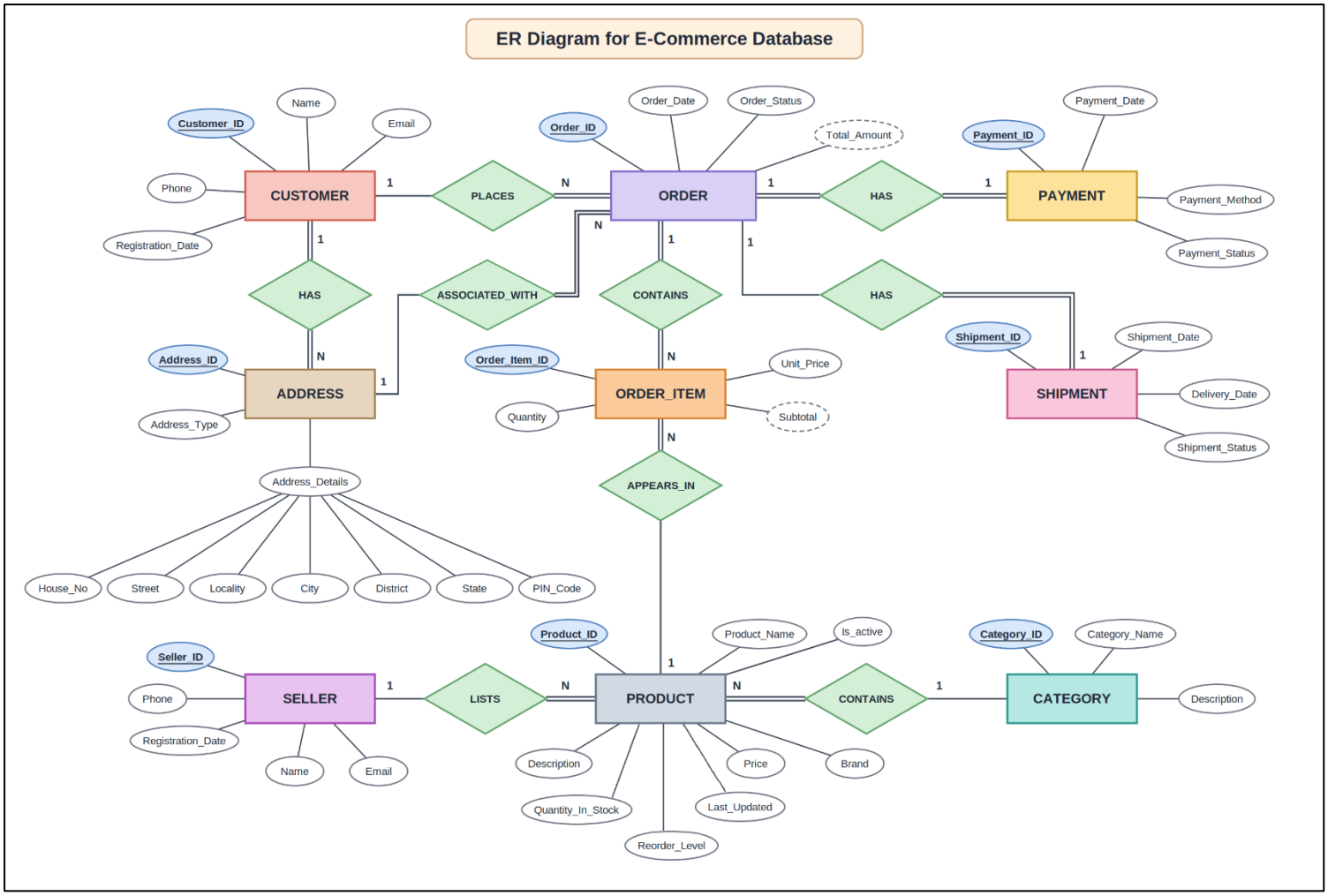
E-Commerce Marketplace Database Design

1. Introduction:

We are designing an E-Commerce platform, which allows sellers to list their products and customers to browse and place orders, much like Amazon. The application design is detailed in the Application Design document.

2. ER Model:

The model covers customers and their addresses, sellers and product listings, categories, orders and order items, payments, and shipments.



3. Database Implementation:

We have decided to use Neon to host our database. It is a cloud-based PostgreSQL implementation, which allows all our team members to access the database simultaneously. All updates to the database are reflected instantly.

4. Relational Schema:

The 9 tables along with their attributes, keys, datatypes and constraints are listed below:

CUSTOMER

Attribute	Datatype	Constraints
customer_id	BIGSERIAL	PK
name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
phone	VARCHAR(15)	NOT NULL, UNIQUE; 10–15 digits
registration_date	DATE	NOT NULL, DEFAULT CURRENT_DATE

ADDRESS

Attribute	Datatype	Constraints
address_id	BIGSERIAL	PK
customer_id	BIGINT	FK → CUSTOMER(customer_id), NOT NULL
address_type	VARCHAR(30)	NOT NULL
house_no	VARCHAR(50)	NOT NULL
street	VARCHAR(150)	NOT NULL
locality	VARCHAR(150)	NOT NULL
city	VARCHAR(100)	NOT NULL
district	VARCHAR(100)	NOT NULL
state	VARCHAR(100)	NOT NULL
pin_code	CHAR(6)	NOT NULL; exactly 6 digits

SELLER

Attribute	Datatype	Constraints
seller_id	BIGSERIAL	PK
name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
phone	VARCHAR(15)	NOT NULL, UNIQUE; 10–15 digits
registration_date	DATE	NOT NULL, DEFAULT CURRENT_DATE

CATEGORY

Attribute	Datatype	Constraints
category_id	BIGSERIAL	PK
category_name	VARCHAR(100)	NOT NULL, UNIQUE
description	TEXT	NULL allowed

PRODUCT

Attribute	Datatype	Constraints
product_id	BIGSERIAL	PK
seller_id	BIGINT	FK → SELLER(seller_id), NOT NULL
category_id	BIGINT	FK → CATEGORY(category_id), NOT NULL
product_name	VARCHAR(200)	UNIQUE(seller_id, product_name), NOT NULL
description	TEXT	NULL allowed
brand	VARCHAR(100)	NULL allowed
price	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0
quantity_in_stock	INTEGER	NOT NULL, DEFAULT 0, CHECK ≥ 0
reorder_level	INTEGER	NOT NULL, DEFAULT 10, CHECK ≥ 0
last_updated	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

ORDERS

Attribute	Datatype	Constraints
order_id	BIGSERIAL	PK
customer_id	BIGINT	FK → CUSTOMER(customer_id), NOT NULL
order_date	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
order_status	VARCHAR(30)	NOT NULL, DEFAULT 'Pending'; CHECK: Pending, Confirmed, Shipped, Delivered, Cancelled
total_amount	NUMERIC(12,2)	NOT NULL, DEFAULT 0, CHECK ≥ 0; maintained by trigger
address_id	BIGINT	FK → ADDRESS(address_id), NOT NULL

ORDER_ITEM

Attribute	Datatype	Constraints
order_item_id	BIGSERIAL	PK
order_id	BIGINT	FK → ORDERS(order_id), NOT NULL
product_id	BIGINT	FK → PRODUCT(product_id), NOT NULL
quantity	INTEGER	NOT NULL, CHECK > 0
unit_price	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0
subtotal	NUMERIC(12,2)	STORED GENERATED: quantity × unit_price

PAYMENT

Attribute	Datatype	Constraints
payment_id	BIGSERIAL	PK
order_id	BIGINT	FK → ORDERS(order_id), NOT NULL, UNIQUE
payment_date	TIMESTAMP	NULL allowed
payment_method	VARCHAR(50)	NOT NULL
payment_status	VARCHAR(30)	NOT NULL, DEFAULT 'Pending'; CHECK: Pending, Successful, Failed, Refunded

SHIPMENT

Attribute	Datatype	Constraints
shipment_id	BIGSERIAL	PK
order_id	BIGINT	FK → ORDERS(order_id), NOT NULL, UNIQUE
shipment_date	TIMESTAMP	NULL allowed
delivery_date	TIMESTAMP	NULL allowed, CHECK: delivery_date >= shipment_date
shipment_status	VARCHAR(30)	NOT NULL, DEFAULT 'Pending'; CHECK: Pending, Shipped, In Transit, Delivered, Cancelled

The BIGSERIAL datatype has been used for primary keys to generate unique, auto-incrementing identifiers and accommodate a large number of records. Also, BIGINT has been used for foreign keys referencing BIGSERIAL primary keys, ensuring datatype compatibility.

5. Triggers:

- Product Timestamp Trigger: Automatically updates the last_updated attribute whenever a product record is modified.
- Order Total Trigger: Recalculates the total amount of an order whenever an associated order item is inserted, updated, or deleted.
- Inventory Management Trigger: Automatically adjusts product stock quantities when order items are added, modified, or removed, helping maintain accurate inventory levels.

6. Assumptions/Business Rules:

i. A customer can save multiple addresses and place multiple orders. Each address and order belongs to a single customer.
ii. A seller can list multiple products, while each product belongs to one seller and one category. A category can contain multiple products.
iii. An order can contain multiple products through the ORDER_ITEM table. Each order item refers to one product and records its quantity and purchase-time unit price.
iv. Each order has exactly one payment, created as Pending at order placement, and at most one shipment.
v. The stock quantity is maintained at the product level within the PRODUCT table itself.
vi. The unit price stored in ORDER_ITEM represents the price at the time of purchase. Any subsequent change in the product's current price will be reflected in the PRODUCT table & should not affect previous orders.

vii. The subtotal is calculated automatically from the quantity and unit price. The order total is maintained using a database trigger that sums the subtotals of its order items.

viii. A payment may be pending, successful, failed or refunded, while an order and shipment can have their respective statuses to track progress through the purchasing and delivery process.

ix. Products referenced by past orders are not deleted. They are delisted via is_active = FALSE

7. Design Justification/Normalisation:

Our E-Commerce database largely follows the principles of the Third Normal Form (3NF) by separating entities such as Customer, Seller, Product, Category, Orders, and Payment into distinct relations, avoiding duplication. There are primary and foreign keys to maintain entity integrity and referential integrity between tables. The ORDER_ITEM relation resolves the many-to-many relationship between orders and products while storing the quantity and purchase-time unit price for each product line. The subtotal is generated from the quantity and the unit price of the product.

We made 3 deliberate exceptions for practical purposes:

- ADDRESS stores city, district and state alongside PIN code because addresses are entered and used as a single unit
- ORDERS stores total_amount, maintained by a trigger, to avoid recomputing it on every read
- Although an address_id belongs to only 1 customer_id, we still keep customer_id in ORDERS because the composite FK (address_id, customer_id) needs it to guarantee that the delivery address belongs to the ordering customer

Overall, this design reduces unnecessary duplication, improves data consistency, and simplifies database maintenance.